

■ MOBILEWRIGHT

End-to-End Mobile Automation Framework

Playwright for iOS & Android · Real Devices & Simulators · AI-Agent Ready
v0.0.24 · Apache 2.0 License · github.com/mobile-next/mobilewright

■ PURPOSE	■ TECH STACK	■ PLATFORMS
Mobile app testing & automation framework	TypeScript / Node.js WebSocket JSON-RPC	iOS · Android Simulators · Real Devices

Overview

Mobilewright is an open-source end-to-end mobile automation framework inspired by Playwright's architecture and developer experience. Built by the mobile-next team, it provides a unified, auto-waiting API for testing and automating both iOS and Android apps across real devices, simulators, and emulators — all from a single TypeScript/JavaScript API.

Every AI agent that controls a browser — browser-use, Claude, Cursor — uses Playwright under the hood. Mobilewright brings the same paradigm to native mobile. No XPath. No coordinates. No vision model needed.

Key Value Propositions

■ Playwright-familiar API — getByRole, getByLabel, expect — zero re-learning if you know Playwright.	■ Zero-config setup — auto-discovers booted simulators and emulators out of the box.
■ True cross-platform — one unified API for both iOS and Android; no platform-specific code branches.	■ Auto-waiting everywhere — every action waits for elements to be visible, enabled, and stable.
■ AI-agent first — exposes the device accessibility tree for deterministic, token-efficient agent control.	■ Cloud-ready — native integration with mobile-use.com for hundreds of real devices in the cloud.

How It Compares

Feature	Mobilewright	Appium	Detox	XCTest/Espresso
API Style	Playwright-style	Selenium/WebDriver	Custom	Native framework
Auto-wait	■ Built-in	■ Manual	■■ Partial	■ Manual
Setup	npm install	Server + drivers	RN only	Xcode/AS only
Cross-platform	■ iOS + Android	■ Verbose	RN only	Single platform
AI agent support	■ First-class	■■ Limited	■ No	■ No
Cloud devices	Via mobile-use.com	Yes (complex)	Simulators only	Yes
Locators	Roles + Labels	XPath, CSS, ID	Test IDs	Native queries

Get Running in 60 Seconds

Prerequisites

- Node.js $\geq$ 18
- A booted iOS simulator, Android emulator, or connected real device
- Xcode (for iOS) or Android SDK + ADB (for Android)

Step 1 — Install

```
# Install the framework  
npm install mobilewright
```

```
# Or scaffold a fresh project instantly  
npm init mobilewright
```

Step 2 — Verify your environment

Run the doctor command to check all dependencies:

```
npx mobilewright doctor  
  
# Add --json for machine-readable output  
npx mobilewright doctor --json
```

The doctor checks Xcode, Android SDK, simulators, ADB, and other dependencies — and tells you exactly what's missing and how to fix it.

Step 3 — Write your first test

```
import { ios, expect } from 'mobilewright';  
  
const device = await ios.launch({ bundleId: 'com.example.myapp' });  
const { screen } = device;  
  
// Fill in login form  
await screen.getByLabel('Email').fill('user@example.com');  
await screen.getByLabel('Password').fill('password123');  
await screen.getByRole('button', { name: 'Sign In' }).tap();  
  
// Assert navigation succeeded  
await expect(screen.getByText('Welcome back')).toBeVisible();  
  
// Take a screenshot  
const screenshot = await screen.screenshot();
```

```
await device.close();
```

Step 4 — Run tests

Command	Description
<code>npx mobilewright test</code>	Run all tests
<code>npx mobilewright test login.test.ts</code>	Run a specific file
<code>npx mobilewright test --grep 'sign in'</code>	Filter by test name
<code>npx mobilewright test --reporter html</code>	Generate HTML report
<code>npx mobilewright test --retries 2</code>	Retry flaky tests
<code>npx mobilewright test --workers 4</code>	Parallel workers
<code>npx mobilewright test --list</code>	List tests without running

mobilewright.config.ts

Create a mobilewright.config.ts in your project root. Config values serve as defaults — LaunchOptions passed directly to ios.launch() always take precedence.

```
import { defineConfig } from 'mobilewright';

export default defineConfig({
  platform: 'ios', // 'ios' | 'android'
  bundleId: 'com.example.myapp', // App bundle ID
  deviceName: 'iPhone 16', // Device name (RegExp supported)
  timeout: 10_000, // Global locator timeout (ms)
  testDir: 'tests', // Directory for test files
  testMatch: '**/*.test.ts', // Glob pattern for test files
  reporter: 'html', // 'list' | 'html' | 'json' | 'junit'
  retries: 2, // Max retries for flaky tests
});
```

All Configuration Options

Option	Type	Description
platform	'ios' 'android'	Device platform
bundleId	string	App bundle / package ID
deviceId	string	Explicit device UDID (skips discovery)
deviceName	RegExp	RegExp to match device name
timeout	number	Global locator timeout in ms
testDir	string	Directory to search for test files
testMatch	string RegExp Array	Glob/RegExp patterns for test files
reporter	string Array	'list' 'html' 'json' 'junit'
retries	number	Maximum retry count for flaky tests
projects	Array	Multi-device/platform project matrix

Multi-Project Config (parallel matrix)

```
export default defineConfig({
  projects: [
    { name: 'iOS', platform: 'ios', bundleId: 'com.app.ios' },
    { name: 'Android', platform: 'android', bundleId: 'com.app.android' },
  ],
});
```

```
1,  
});
```

Launchers — ios & android

The top-level entry points, analogous to Playwright's chromium/firefox/webkit. `launch()` handles the full lifecycle: checking mobilecli, discovering devices, connecting, optionally launching the app, and graceful shutdown.

```
import { ios, android } from 'mobilewright';

const device = await ios.launch(); // auto-discovery
const device = await ios.launch({ bundleId: 'com.app' }); // launch specific app
const device = await ios.launch({ deviceName: /iPhone 16/ });
const device = await ios.launch({ deviceId: '5A5FCFCA-...' }); // explicit UDID

const devices = ios.devices(); // list available iOS devices
const devices = android.devices(); // list available Android devices
```

Screen — Locator Factories

Method	Finds by	Example
<code>getByLabel(text)</code>	Accessibility label	<code>screen.getByLabel('Email')</code>
<code>getByRole(role)</code>	Semantic role	<code>screen.getByRole('button', { name: 'OK' })</code>
<code>getByText(text)</code>	Visible text	<code>screen.getByText('Welcome')</code>
<code>getByText(/regex/)</code>	RegExp match	<code>screen.getByText(/welcome/i)</code>
<code>getByTestId(id)</code>	Accessibility ID	<code>screen.getByTestId('login-btn')</code>
<code>getByType(type)</code>	Element type	<code>screen.getByType('TextField')</code>

Screen — Direct Actions

Action	Usage
<code>screenshot()</code>	<code>await screen.screenshot()</code>
<code>screenshot(options)</code>	<code>await screen.screenshot({ format: 'jpeg', quality: 80 })</code>
<code>swipe(direction)</code>	<code>await screen.swipe('up')</code>
<code>swipe(dir, options)</code>	<code>await screen.swipe('down', { distance: 300, duration: 500 })</code>
<code>pressButton(key)</code>	<code>await screen.pressButton('HOME')</code>
<code>tap(x, y)</code>	<code>await screen.tap(195, 400)</code>

Locator Actions

Locators are lazy and chainable — no queries execute until you call an action or assertion. All actions auto-wait for the element to be visible, enabled, and have stable bounds.

Action	Description
<code>locator.tap()</code>	Tap the element
<code>locator.doubleTap()</code>	Double-tap the element
<code>locator.longPress(opts)</code>	Long press (default: { duration: 1000 ms })
<code>locator.fill(text)</code>	Tap to focus then type text (clears existing)
<code>locator.isVisible()</code>	Returns boolean — is element in the view hierarchy?
<code>locator.isEnabled()</code>	Returns boolean — is element interactable?
<code>locator.isSelected()</code>	Returns boolean — is element selected/active?
<code>locator.isFocused()</code>	Returns boolean — does element have focus?
<code>locator.isChecked()</code>	Returns boolean — is checkbox/toggle checked?
<code>locator.getText()</code>	Waits for visibility then returns text content
<code>locator.getValue()</code>	Returns raw value (e.g. text field content)
<code>locator.waitFor({state})</code>	Explicitly wait: 'visible' 'hidden' 'enabled' 'disabled'

Chaining Locators

Scope queries within a parent element's bounds. Child lookups use bounds-based containment.

```
// Tap delete button inside the first list row
const row = screen.getByType('Cell');
await row.getByRole('button', { name: 'Delete' }).tap();

// Get text from navigation bar
const title = await screen
  .getByType('NavigationBar')
  .getByType('StaticText')
  .getText();
```

Assertions — expect()

All assertions poll repeatedly until satisfied or timeout (default 5 s). Every assertion supports `.not` for negation.

Assertion	Description
<code>expect(l).toBeVisible()</code>	Element is in the accessibility tree and visible
<code>expect(l).not.toBeVisible()</code>	Element is hidden or absent
<code>expect(l).toBeEnabled()</code>	Element can be interacted with
<code>expect(l).toHaveText('...')</code>	Exact or RegExp text match
<code>expect(l).toContainText('.')</code>	Element text contains substring
<code>expect(l).toBeVisible({timeout: 10_000})</code>	Custom timeout override
<code>expect(l).toBeChecked()</code>	Checkbox or toggle is checked
<code>expect(l).toMatchObject({...})</code>	Matches object shape (structural)
<code>expect(l).toHaveLength(n)</code>	Array or collection length check

Device — Lifecycle & Controls

Method	Description
device.setOrientation('landscape')	Rotate device
device.getOrientation()	Get current orientation
device.goto('myapp://settings')	Open deep link (Playwright-style alias)
device.openUrl('https://...')	Open a URL
device.launchApp(bundleId, opts)	Launch app (with optional locale, etc.)
device.terminateApp(bundleId)	Kill the app
device.listApps()	List installed apps
device.getForegroundApp()	Get currently active app
device.installApp('/path/app.ipa')	Install an IPA or APK
device.uninstallApp(bundleId)	Uninstall an app
device.close()	Disconnect and stop auto-started mobilecli

getByRole — Cross-Platform Mapping

Role	iOS Types	Android Types
button	Button, ImageButton	Button, ImageButton, ReactViewGroup*
textfield	TextField, SecureTextField, SearchField	EditText, ReactEditText
text	StaticText	TextView, Text, ReactTextView
image	Image	ImageView, ReactImageView
switch	Switch	Switch, Toggle
checkbox	—	Checkbox
slider	Slider	SeekBar
list	Table, CollectionView, ScrollView	ListView, RecyclerView, ReactScrollView
header	NavigationBar	Toolbar, Header
link	Link	Link
listitem	Cell	LinearLayout, RelativeLayout, Other
tab	Tab, TabBar	Tab, TabBar

* ReactViewGroup matches 'button' only when the element has clickable='true' or accessible='true' in its attributes, to avoid false positives in React Native containers. Falls back to direct type matching if no role mapping exists.

@mobilewright/test — Vitest / Playwright Fixtures

@mobilewright/test extends Playwright Test with mobile-specific fixtures. The device fixture connects once per worker and the screen fixture provides device.screen to each test, with automatic screenshot-on-failure and optional video recording.

```
import { test, expect } from '@mobilewright/test';

// Configure for all tests in this file
test.use({ bundleId: 'com.example.myapp', video: 'on' });

test('can sign in', async ({ device, screen, bundleId }) => {
  // Fresh-launch before each test
  await device.terminateApp(bundleId).catch(() => {});
  await device.launchApp(bundleId);

  await screen.getByLabel('Email').fill('user@example.com');
  await screen.getByLabel('Password').fill('password123');
  await screen.getByRole('button', { name: 'Sign In' }).tap();

  await expect(screen.getByText('Welcome back')).toBeVisible();
});
```

Package Architecture

Package	Purpose
mobilewright	Main entry — ios/android launchers, expect, config, CLI
@mobilewright/test	Test fixtures for Playwright Test / Vitest
@mobilewright/protocol	TypeScript interfaces: MobilewrightDriver, ViewNode
@mobilewright/driver-mobilecli	WebSocket JSON-RPC client for local mobilecli
@mobilewright/driver-mobile-use	WebSocket JSON-RPC client for mobile-use.com cloud
@mobilewright/mobilewright-core	Device, Screen, Locator, expect — core user-facing API

Most users only need mobilewright (or @mobilewright/test for Vitest integration). The internal packages are published separately for embedding in custom frameworks.

Supported Mobile Frameworks

Framework	iOS	Android	Notes
UIKit / Storyboards	■	—	Full native element types; all locators work
SwiftUI	■	—	Maps to XCUIElementType accessibility tree
Jetpack Compose	—	■	Renders to native Android accessibility nodes
Android Views (XML)	—	■	Full native element types; all locators work
React Native	■	■	Real native components; RN types mapped to roles
Expo	■	■	Same as React Native (Expo builds to RN)
Flutter	■	■	Requires Dart VM Service driver (in progress)
.NET MAUI	■	■	Compiles to native controls on both platforms
Kotlin Multiplatform	■	■	iOS Compose Multiplatform support in progress
Cordova / Capacitor	■	■	WebView content via native accessibility tree
NativeScript	■	■	Renders to native views on both platforms

Legend

■ Full support	■ In progress	— Not applicable
----------------	---------------	------------------

Built for AI Agents

Mobilewright is designed from the ground up for AI agents. Instead of requiring a vision model or screenshot parsing, it exposes the device's native accessibility tree — deterministic, token-efficient, and readable by any LLM or coding agent.

- Accessibility tree exposure — agents read structured UI data, not raw pixels
- Works with Claude, Cursor, Copilot, browser-use, and any MCP client
- Integrates natively with mobile-mcp (Model Context Protocol server for mobile)
- No vision model needed — semantic roles and labels are human-readable
- Deterministic actions — no flakiness from coordinate-based clicks

Example — AI Agent controlling a real phone

```
// An AI agent can control a real phone with readable, semantic actions
await screen.getByRole('button', { name: 'Sign In' }).tap();
await screen.getByLabel('Email').fill('user@example.com');
await expect(screen.getByText('Welcome')).toBeVisible();

// No XPath. No coordinates. No vision model.
// The agent reads the accessibility tree and acts directly.
```

Cloud Devices — mobile-use.com

mobile-use.com is the only device cloud with native Mobilewright support. It provides API access to hundreds of real Android and iOS devices. Your Mobilewright scripts run with zero modification — just point your config at the mobile-use.com endpoint and go.

```
// mobilewright.config.ts — point to cloud
export default defineConfig({
  platform: 'ios',
  bundleId: 'com.example.app',
  // Use mobile-use.com endpoint instead of local mobilecli
  endpoint: 'wss://api.mobile-use.com/v1',
  apiKey: process.env.MOBILE_USE_API_KEY,
});
```

Command-Line Interface

Command	What it does
npx mobilewright init	Scaffold mobilewright.config.ts + example.test.ts
npx mobilewright devices	List all connected devices, simulators, emulators
npx mobilewright test	Run all tests (auto-discovers config)
npx mobilewright test [file]	Run a specific test file
npx mobilewright test --grep	Filter tests by name pattern
npx mobilewright test --reporter	Set reporter: list html json junit
npx mobilewright test --retries	Set retry count for flaky tests
npx mobilewright test --workers	Set number of parallel workers
npx mobilewright test --list	List tests without executing
npx mobilewright show-report	Open the HTML report in browser
npx mobilewright doctor	Check environment health (Xcode, ADB, etc.)
npx mobilewright doctor --json	Machine-readable environment check output

mobilewright devices — sample output

[illegible]

Debugging Tips

Tip	Command / Note
Enable debug logging	<code>DEBUG=mw:* npx mobilewright test</code>
Verbose driver output	<code>MWDEBUGIMPL=1 npx mobilewright test</code>
Environment check	<code>npx mobilewright doctor --json</code>
Trace clean failures	Stack traces from <code>expect()</code> failures omit mobilewright frames by default

Useful Resources

Resource	URL
GitHub Repository	github.com/mobile-next/mobilewright
Documentation	mobilewright.dev
Cloud Device Lab	mobile-use.com
Mobile MCP Server	github.com/mobile-next/mobile-mcp
Mobile CLI	github.com/mobile-next/mobilecli
npm Package	npmjs.com/package/mobilewright
Latest Release	v0.0.24 (Apache 2.0)
YouTube Intro	Introducing Mobilewright: Playwright for Mobile

License & Contributing

Mobilewright is licensed under the Apache License 2.0. Contributions are welcome — clone the repo, run `npm test`, and submit a pull request. The project is maintained by the mobile-next team and is actively developed with regular releases.

To contribute: fork → branch → `npm test` → PR. Issues and feature requests are tracked at github.com/mobile-next/mobilewright/issues.